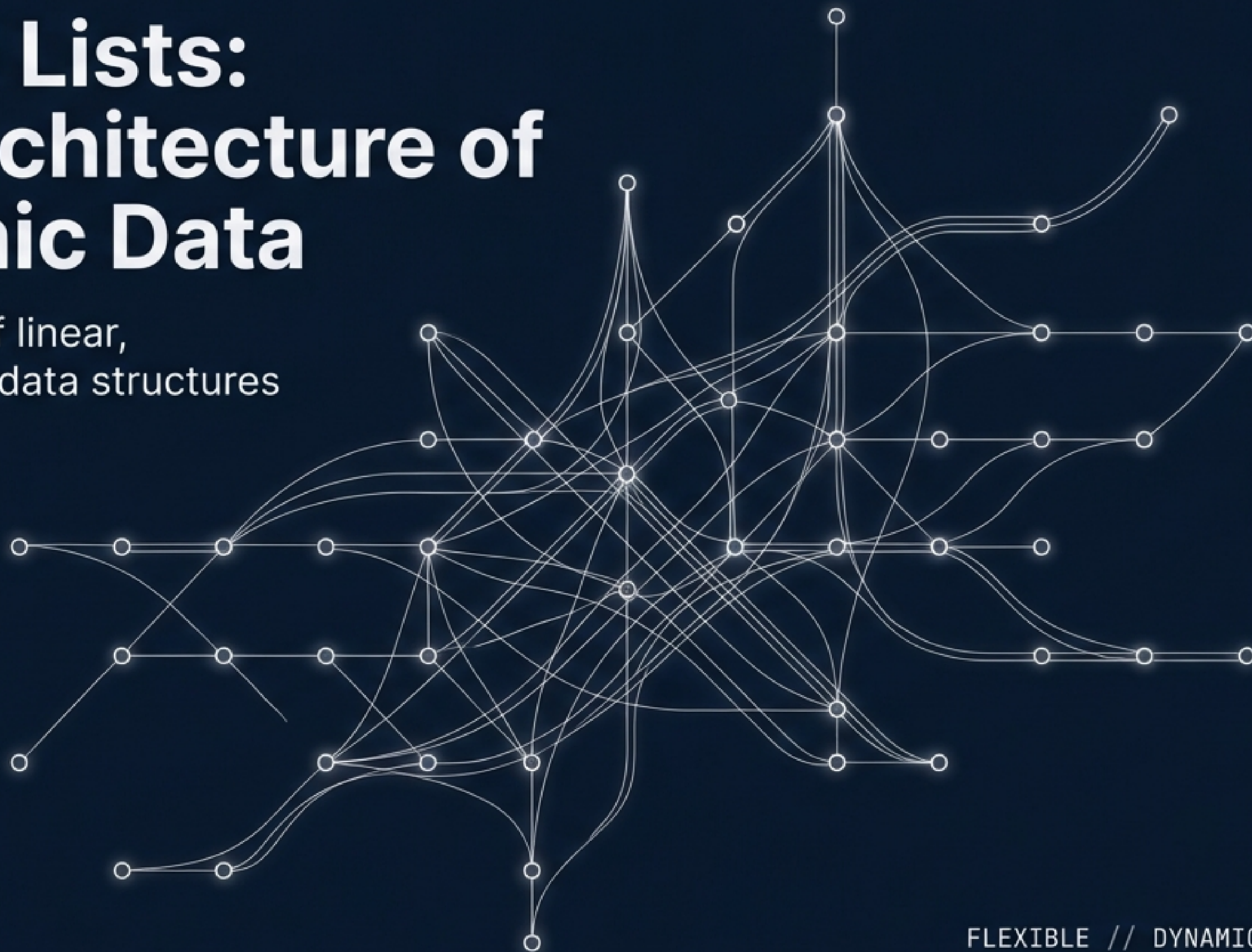


Linked Lists: The Architecture of Dynamic Data

A visual atlas of linear,
self-referential data structures



FLEXIBLE // DYNAMIC // NON-CONTIGUOUS

The Atomic Unit: Anatomy of a Node



```
struct node {  
    int data;  
    struct node *next;  
};
```

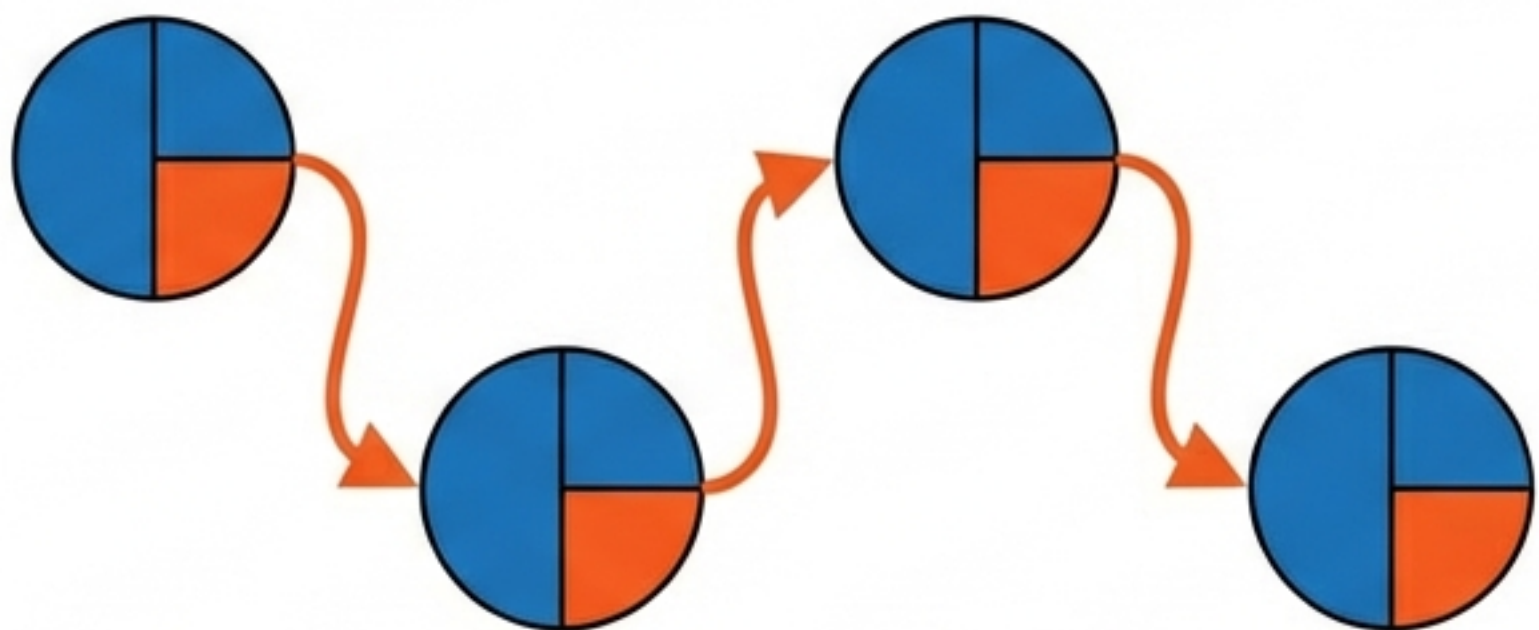
Unlike arrays, nodes are 'self-referential'. They contain data and the map to find the next piece of data.

Static Rigidity vs. Dynamic Flexibility

The Array (Static)

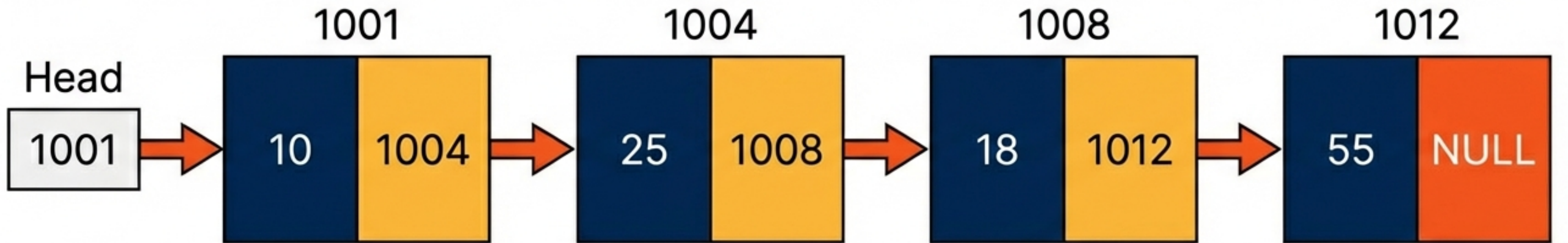


The Linked List (Dynamic)



Feature	Array	Linked List
Size	Fixed (Compile time)	Dynamic (Runtime)
Memory	Contiguous Block	Scattered / Non-contiguous
Insertion	Expensive (Shifting)	Efficient (Pointer update)

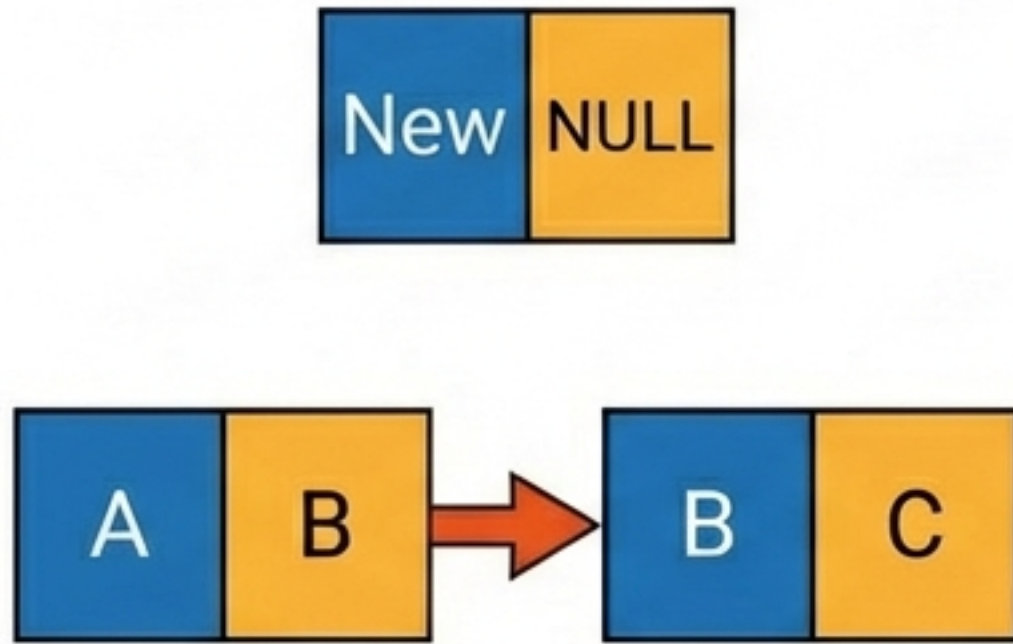
The Singly Linked List (SLL)



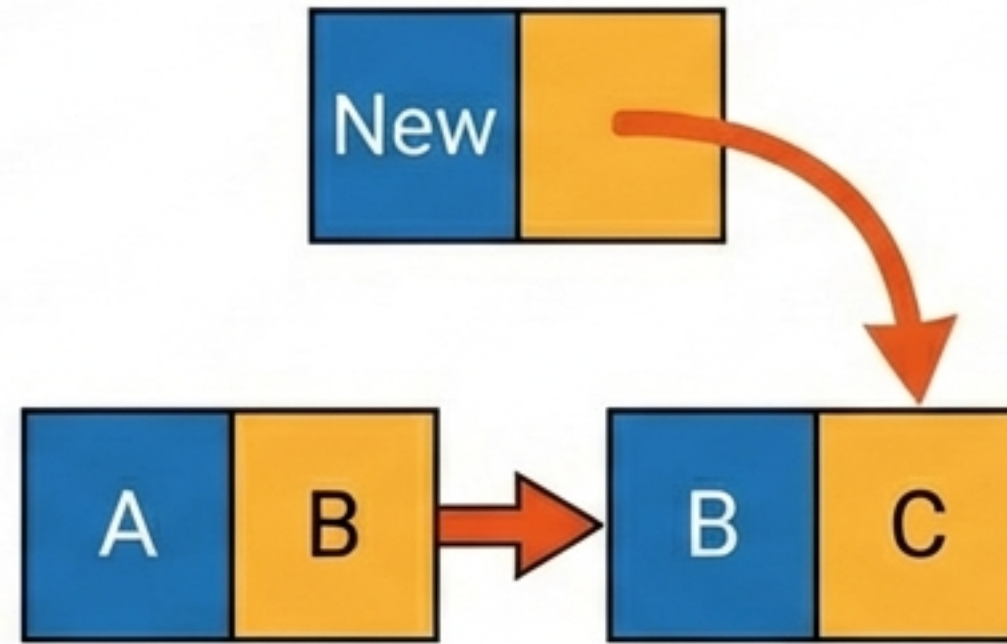
Entry Point: If Head is lost, the list is lost.

Terminator: NULL indicates the end of the chain.

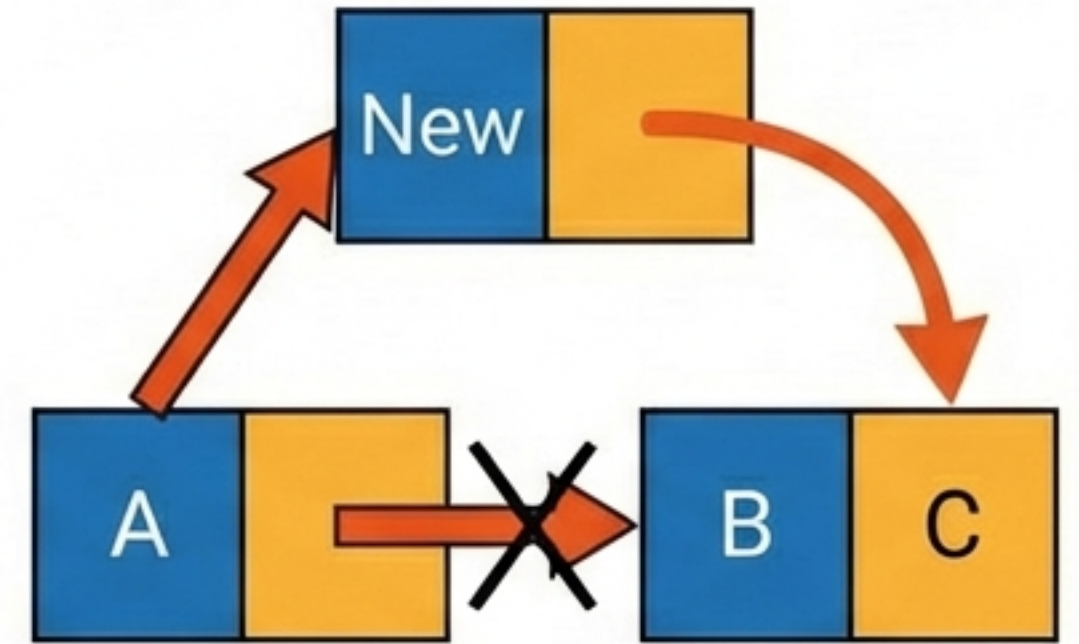
Blueprint: Insertion Logic



1. Allocation



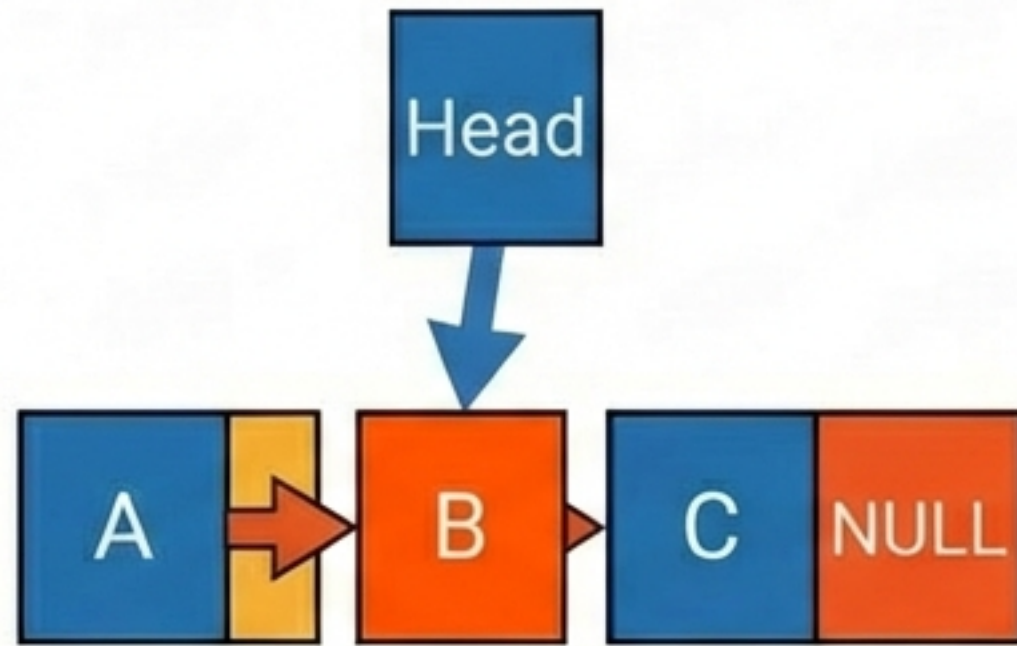
2. Linking Out



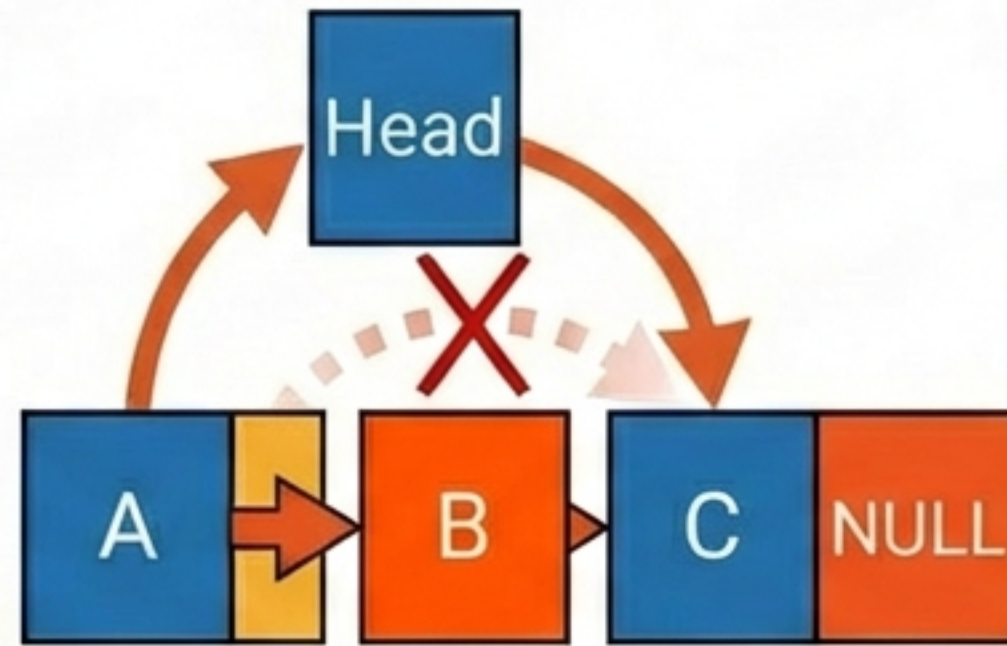
3. Linking In

```
new->next = temp->next;  
temp->next = new;
```

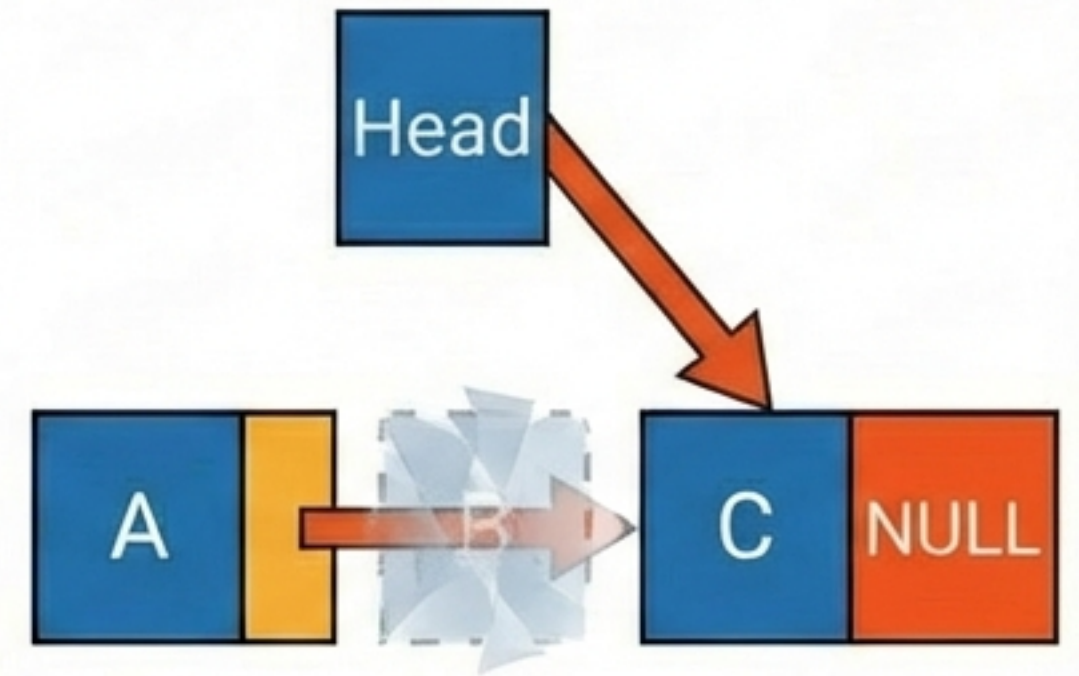

Blueprint: Deletion Logic



1. Identify



2. Bridge



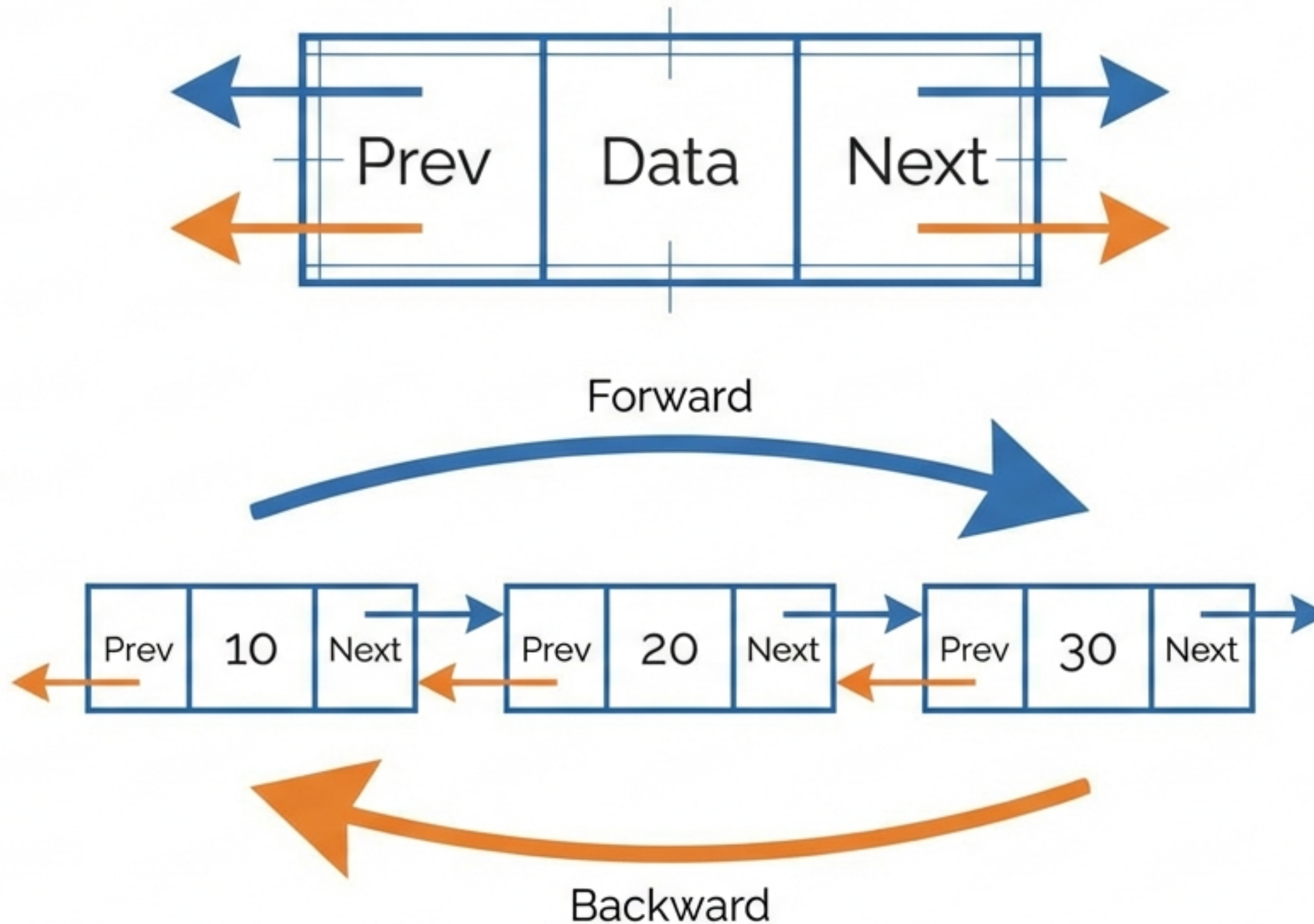
3. Liberate

Bridge the gap before burning the bridge.
`free(temp);`

Doubly Linked Lists: Bidirectional Navigation

PROS

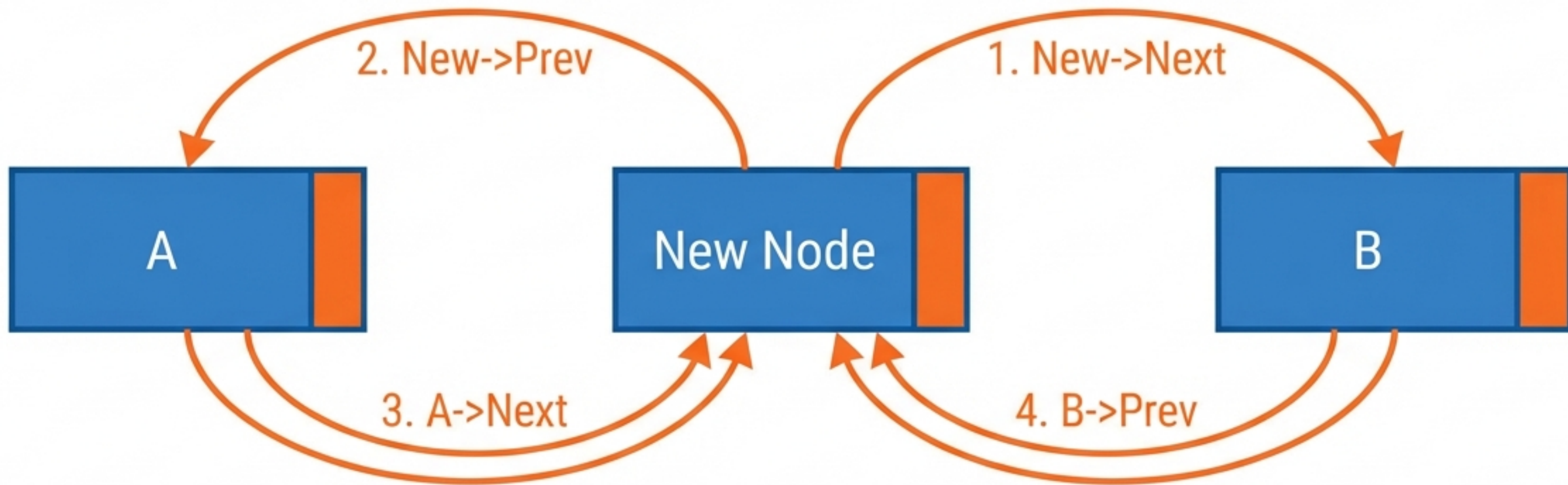
- Bidirectional traversal
- $O(1)$ deletion (if node known)



CONS

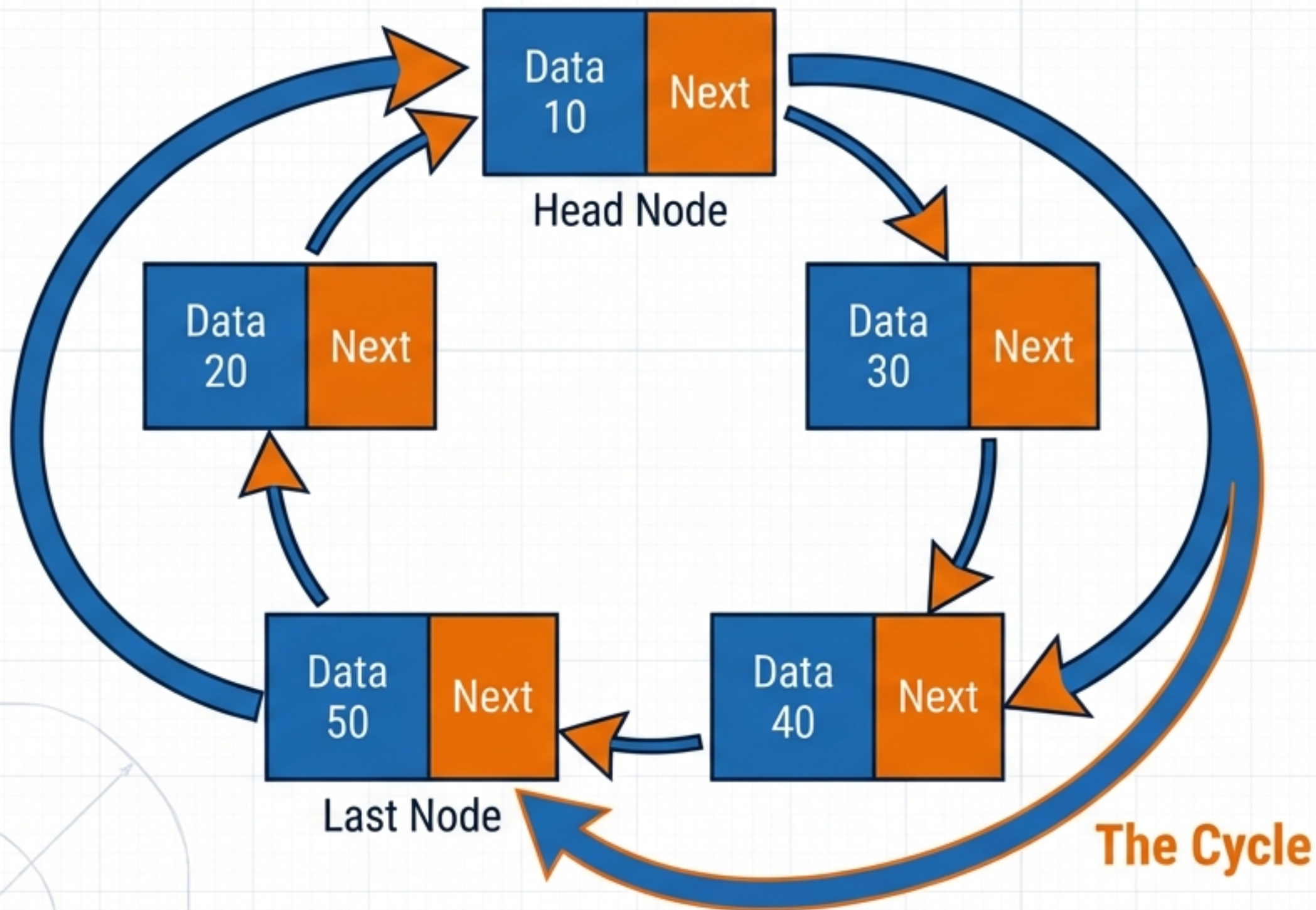
- Higher memory overhead
- Complex pointer management

DLL Operations: The Four-Pointer Handshake



4 pointer updates are required to insert a node without breaking the chain.

Circular Linked Lists: The Infinite Loop



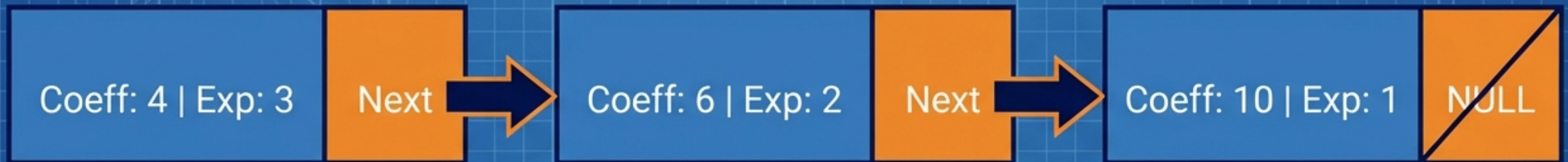
- **No NULL** terminator
- **Any** node can be a starting point
- **Use Cases:** Round-Robin scheduling, Media playlists

Structural Comparison Matrix

Structure	Traversal	Memory Cost	Complexity
Singly Linked List	Forward Only	Low (2 fields)	Simple
Doubly Linked List	Bi-directional	High (3 fields)	Moderate
Circular Linked List	Infinite Loop	Low (2 fields)	Moderate (Loop risk)

Application: Polynomial Representation

$$4x^3 + 6x^2 + 10x$$



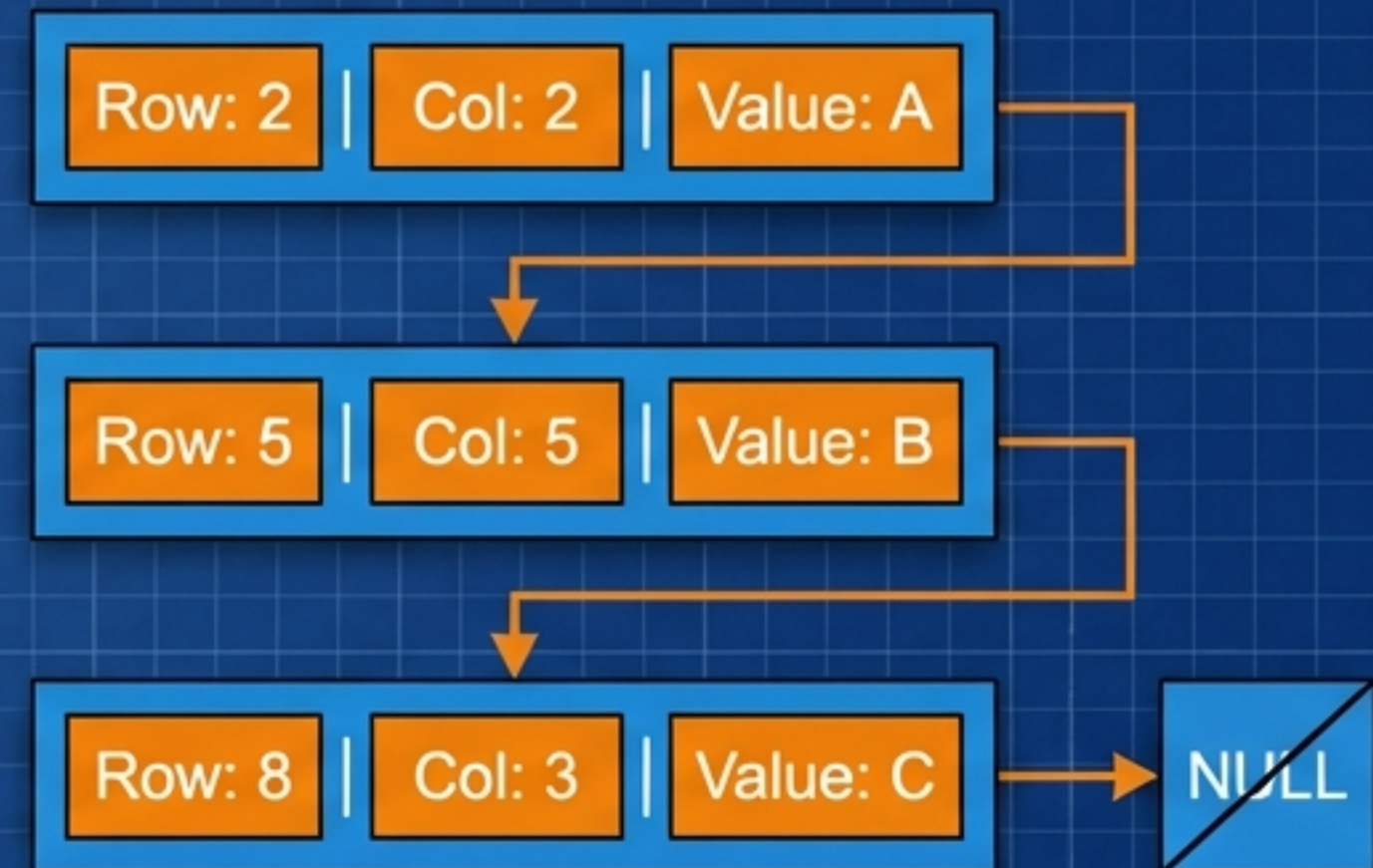
Efficiently stores sparse mathematical data without allocating memory for zero-value terms.

Application: Sparse Matrices

Standard Array (High Waste)

0	0	0	0	0	0	0	0	0	0
0	0	A	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	B	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	C	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

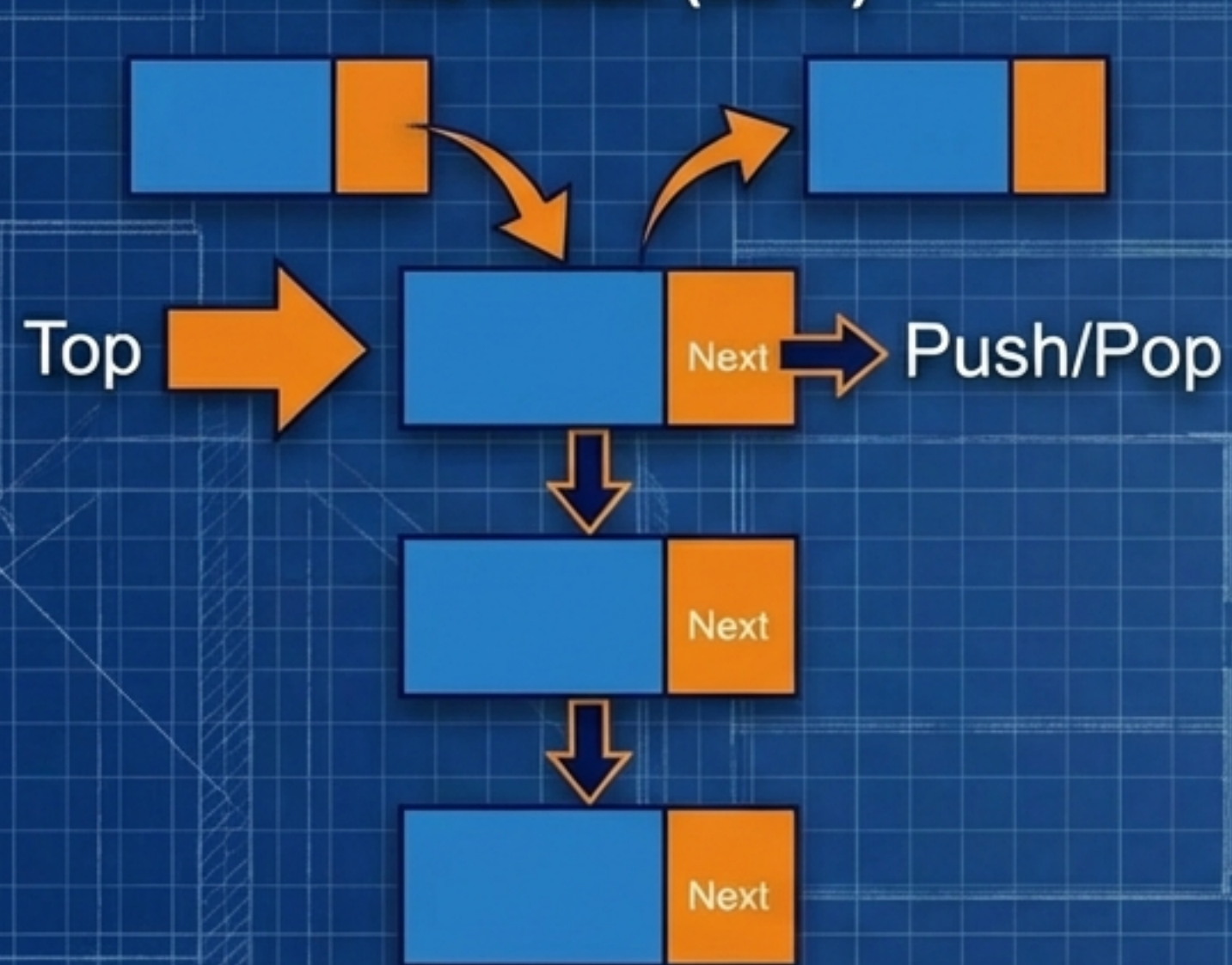
Linked Representation (High Efficiency)



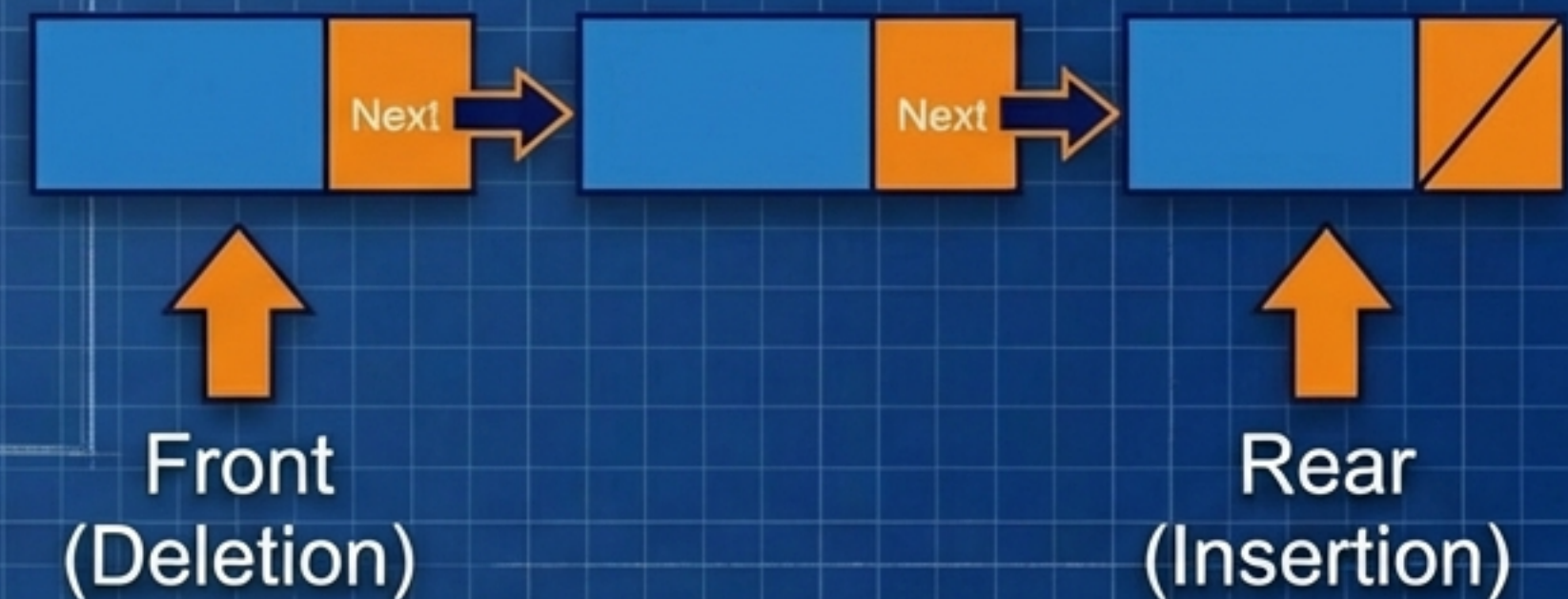
Efficiently stores sparse data by linking only non-zero elements, eliminating memory waste for zeros.

The Engine Room: Stacks & Queues

The Stack (LIFO)



The Queue (FIFO)



Linked Lists allow Stacks and Queues to grow indefinitely, unlike fixed-array implementations.

Top Engineering Challenges

Reverse a List



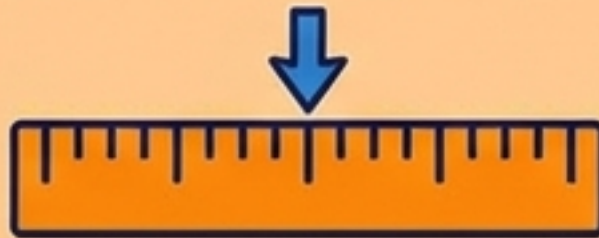
Iterative or Recursive approach.

Detect a Cycle



Floyd's Cycle-Finding Algorithm.

Find Middle Node



Two-pointer technique (Fast/Slow).

Merge Lists



Combine sorted chains.

Executive Summary: The Trade-Offs

The Wins	The Costs
✓ Dynamic Size (Runtime allocation) ✓ Efficient Memory Utilization ✓ Fast Insertions/Deletions	✗ Sequential Access Only (No random access) ✗ Memory Overhead (Pointers) ✗ No cache locality benefit

Choose Linked Lists when data volume is unpredictable and insertion speed is paramount.